REFERENCE

AI Tools and Databases

The AI models the data center registers, the tables and datasets it holds, the tools that operate it, and how its search works.

Software da Database User 13

> Built from the software and its bundled sample data. Model entries, table columns, row counts, command help and portal routes are read when the document is built, not copied by hand.

7	72	24	12	18	11
databases, 13 tables, 1,577 rows	AI models, 79 versions	model providers, 9 tasks	datasets, 462 records	command-line commands	portal routes

1. AI model registry
2. The seven databases in detail
3. Datasets
4. Tools
5. Search

1. AI model registry

The `models` database registers the AI models the data center can call or host. Its `models` table has one row per model name, with its provider and task; `model_versions` has one row per version and marks which are active. The registry holds 72 models from 24 providers, in 79 versions across nine tasks. 72 of the 79 versions are active. The other seven are inactive, each kept beside an active version of the same model.

By task

Task	Models	Versions	Active
Embedding	17	17	17
Chat	18	23	18
Rerank	6	6	6
Speech to text	6	7	6
Text to speech	5	5	5
Image generation	6	6	6
Vision	5	5	5
Code	4	5	4
Moderation	5	5	5
Total	**72**	**79**	**72**

Embedding · 17 models, 17 versions, 17 active

Turn text into vectors, for semantic search, clustering and retrieval. Dimensions is the length of the vector a model returns; the registry records it for these models only.

Model	Provider	Version	Dimensions	Active
`BAAI/bge-large-en-v1.5`	BAAI	1.5	1,024	yes
`Qwen/Qwen3-Embedding-8B`	Alibaba	3	4,096	yes
`embed-english-v3.0`	Cohere	3.0	1,024	yes
`embed-multilingual-v3.0`	Cohere	3.0	1,024	yes
`embed-v4.0`	Cohere	4.0	1,536	yes
`gemini-embedding-001`	Google	001	3,072	yes
`jinaai/jina-embeddings-v3`	Jina AI	3	1,024	yes
`local-hashing-v1`	built-in	1.0.0	4,096*	yes
`nomic-ai/nomic-embed-text-v1.5`	Nomic	1.5	768	yes
`nvidia/NV-Embed-v2`	NVIDIA	2	4,096	yes
`sentence-transformers/all-MiniLM-L6-v2`	Sentence-Transformers	2	384	yes
`sentence-transformers/all-mpnet-base-v2`	Sentence-Transformers	2	768	yes
`text-embedding-004`	Google	004	768	yes
`text-embedding-3-large`	OpenAI	3	3,072	yes
`text-embedding-3-small`	OpenAI	3	1,536	yes
`voyage-3-large`	Voyage AI	3	1,024	yes
`voyage-3.5`	Voyage AI	3.5	1,024	yes

* `local-hashing-v1` is the built-in embedder that indexes the stored documents (section 5). Its registry entry lists 4,096 dimensions, but its vectors are sparse: the `embeddings` table records the bucket space they were hashed into, 9,223,372,036,854,775,807 ($2^{63} - 1$), and each vector stores only the buckets its text uses.

Chat · 18 models, 23 versions, 18 active

General-purpose language models that answer, summarise and reason over text.

Model	Provider	Version	Active
`Qwen/Qwen3-235B-A22B`	Alibaba	3	yes
`amazon.nova-pro-v1:0`	Amazon	1	yes
`claude-haiku-4-5`	Anthropic	4.5	yes
`claude-opus-5`	Anthropic	5	yes
`claude-sonnet-5`	Anthropic	5	yes
`command-a-03-2025`	Cohere	03-2025	yes
`command-r-plus`	Cohere	04-2024	no
`command-r-plus`	Cohere	08-2024	yes
`deepseek-chat`	DeepSeek	V3	no
`deepseek-chat`	DeepSeek	V3-0324	yes

Model	Provider	Version	Active
`deepseek-reasoner`	DeepSeek	R1	no
`deepseek-reasoner`	DeepSeek	R1-0528	yes
`gemini-2.5-pro`	Google	2.5	yes
`gpt-4.1`	OpenAI	gpt-4.1-2025-04-14	yes
`gpt-5`	OpenAI	gpt-5-2025-08-07	yes
`grok-4`	xAI	grok-4-0709	yes
`meta-llama/Llama-3.3-70B-Instruct`	Meta	3.3	yes
`meta-llama/Llama-4-Maverick-17B-128E-Instruct`	Meta	4	yes
`microsoft/Phi-4`	Microsoft	4	yes
`mistral-large-latest`	Mistral	mistral-large-2407	no
`mistral-large-latest`	Mistral	mistral-large-2411	yes
`mistral-small-latest`	Mistral	mistral-small-2503	no
`mistral-small-latest`	Mistral	mistral-small-2506	yes

Rerank · 6 models, 6 versions, 6 active

Score a short list of candidate passages against a query, to reorder search results.

Model	Provider	Version	Active
`BAAI/bge-reranker-v2-m3`	BAAI	2	yes
`Qwen/Qwen3-Reranker-8B`	Alibaba	3	yes
`cross-encoder/ms-marco-MiniLM-L-6-v2`	Sentence-Transformers	2	yes
`jinaai/jina-reranker-v2-base-multilingual`	Jina AI	2	yes
`rerank-2`	Voyage AI	2	yes
`rerank-v3.5`	Cohere	3.5	yes

Speech to text · 6 models, 7 versions, 6 active

Transcribe recorded or streamed audio into text.

Model	Provider	Version	Active
`facebook/seamless-m4t-v2-large`	Meta	2	yes
`nova-3`	Deepgram	3	yes
`nvidia/parakeet-tdt-0.6b-v2`	NVIDIA	2	yes
`openai/whisper`	OpenAI	large-v3	no
`openai/whisper`	OpenAI	large-v3-turbo	yes
`scribe_v1`	ElevenLabs	1	yes
`whisper-1`	OpenAI	1	yes

Text to speech · 5 models, 5 versions, 5 active

Read text aloud as synthesised speech.

Model	Provider	Version	Active
`coqui/XTTS-v2`	Coqui	2	yes
`eleven_multilingual_v2`	ElevenLabs	2	yes
`eleven_v3`	ElevenLabs	3	yes
`gemini-2.5-flash-preview-tts`	Google	2.5	yes
`tts-1`	OpenAI	1	yes

Image generation · 6 models, 6 versions, 6 active

Create images from a text prompt.

Model	Provider	Version	Active
`black-forest-labs/FLUX.1-dev`	Black Forest Labs	1	yes
`dall-e-3`	OpenAI	3	yes
`gpt-image-1`	OpenAI	1	yes
`imagen-3.0-generate-002`	Google	002	yes
`stabilityai/stable-diffusion-3.5-large`	Stability AI	3.5	yes
`stabilityai/stable-diffusion-xl-base-1.0`	Stability AI	1.0	yes

Vision · 5 models, 5 versions, 5 active

Take images alongside text: describe, caption, answer questions and read text in them.

Model	Provider	Version	Active
`Qwen/Qwen2.5-VL-72B-Instruct`	Alibaba	2.5	yes
`google/paligemma2-3b-pt-224`	Google	2	yes
`meta-llama/Llama-3.2-11B-Vision-Instruct`	Meta	3.2	yes
`microsoft/Florence-2-large`	Microsoft	2	yes
`pixtral-large-latest`	Mistral	pixtral-large-2411	yes

Code · 4 models, 5 versions, 4 active

Language models tuned to write, complete and explain source code.

Model	Provider	Version	Active
`Qwen/Qwen2.5-Coder-32B-Instruct`	Alibaba	2.5	yes
`bigcode/starcoder2-15b`	BigCode	2	yes
`codestral-latest`	Mistral	codestral-2405	no
`codestral-latest`	Mistral	codestral-2501	yes
`deepseek-ai/DeepSeek-Coder-V2-Instruct`	DeepSeek	2	yes

Moderation · 5 models, 5 versions, 5 active

Classify text or images against a content policy before they reach a person or another model.

Model	Provider	Version	Active
`google/shieldgemma-2-4b-it`	Google	2	yes
`meta-llama/Llama-Guard-3-8B`	Meta	3	yes
`meta-llama/Llama-Guard-4-12B`	Meta	4	yes
`mistral-moderation-latest`	Mistral	mistral-moderation-2411	yes
`omni-moderation-latest`	OpenAI	latest	yes

By provider · most versions first; totals as by task

Provider	Models	Versions	Active
OpenAI	10	11	10
Mistral	5	8	5
Cohere	6	7	6
Google	7	7	7
Meta	6	6	6
Alibaba	5	5	5
DeepSeek	3	5	3
Anthropic	3	3	3
ElevenLabs	3	3	3
Sentence-Transformers	3	3	3
Voyage AI	3	3	3
BAAI	2	2	2

Provider	Models	Versions	Active
Jina AI	2	2	2
Microsoft	2	2	2
NVIDIA	2	2	2
Stability AI	2	2	2
Amazon	1	1	1
BigCode	1	1	1
Black Forest Labs	1	1	1
built-in	1	1	1
Coqui	1	1	1
Deepgram	1	1	1
Nomic	1	1	1
xAI	1	1	1

2. The seven databases in detail

Each database is a separate SQLite file under the storage root. The columns below are read from the live schema with `PRAGMA table_info`, so they show what SQLite holds rather than what the source code intends: 13 tables with 75 columns in all. Row counts are those after `python -m datacenter seed` fills an empty root, which loads the bundled data files and runs the jobs that loading queues; together the tables hold 1,577 rows. The audit count grows with every later command.

"Not null" is SQLite's own flag. An `INTEGER PRIMARY KEY` column is SQLite's row id: it reports as nullable, but SQLite assigns the next id when none is given. "References" names the column a foreign key points at, with what happens to this row when that one is deleted. A foreign key cannot reach into another file, so the link from `embeddings.document_id` to `documents` is kept by the `DataCenter` class rather than by SQLite.

1. Contacts · `contacts.db` · 3 tables, 107 rows

People and organizations the business works with.

`organizations` · 20 rows

Column	Type	Not null	Default	Key	References
`id`	INTEGER	no	—	primary	
`name`	TEXT	yes	—		
`industry`	TEXT	no	—		
`website`	TEXT	no	—		
`created_at`	TEXT	yes	`datetime('now')`		

Unique: `(name)`.

`contacts` · 45 rows

Column	Type	Not null	Default	Key	References
`id`	INTEGER	no	—	primary	
`organization_id`	INTEGER	no	—		`organizations.id` on delete set null
`full_name`	TEXT	yes	—		
`role`	TEXT	no	—		
`email`	TEXT	no	—		
`phone`	TEXT	no	—		
`notes`	TEXT	no	—		
`created_at`	TEXT	yes	`datetime('now')`		

Unique: `(email)`. Indexes: `idx_contacts_name (full_name)`; `idx_contacts_org (organization_id)`.

`contact_channels` · 42 rows

Column	Type	Not null	Default	Key	References
`id`	INTEGER	no	—	primary	
`contact_id`	INTEGER	yes	—		`contacts.id` on delete cascade
`channel`	TEXT	yes	—		
`handle`	TEXT	yes	—		
`is_primary`	INTEGER	yes	`0`		

Unique: `(contact_id, channel, handle)`.

2. Documents · `documents.db` · 2 tables, 298 rows

Documents, their text and their metadata.

`documents` · 79 rows

Column	Type	Not null	Default	Key	References
`id`	INTEGER	no	—	primary	
`external_id`	TEXT	no	—		
`title`	TEXT	yes	—		
`body`	TEXT	yes	`''`		
`source`	TEXT	no	—		
`mime_type`	TEXT	yes	`'text/plain'`		
`checksum`	TEXT	no	—		
`owner_email`	TEXT	no	—		
`created_at`	TEXT	yes	`datetime('now')`		
`updated_at`	TEXT	yes	`datetime('now')`		

Unique: `(external_id)`. Index: `idx_documents_title (title)`.

`document_tags` · 219 rows

Column	Type	Not null	Default	Key	References
`document_id`	INTEGER	yes	—	primary, 1 of 2	`documents.id` on delete cascade
`tag`	TEXT	yes	—	primary, 2 of 2	

Index: `idx_document_tags_tag (tag)`.

3. Datasets · `datasets.db` · 2 tables, 474 rows

Structured data records grouped into named datasets.

`datasets` · 12 rows

Column	Type	Not null	Default	Key	References
`id`	INTEGER	no	—	primary	
`name`	TEXT	yes	—		
`description`	TEXT	no	—		
`schema_json`	TEXT	no	—		
`created_at`	TEXT	yes	`datetime('now')`		

Unique: `(name)`.

`records` · 462 rows

Column	Type	Not null	Default	Key	References
`id`	INTEGER	no	—	primary	
`dataset_id`	INTEGER	yes	—		`datasets.id` on delete cascade
`payload`	TEXT	yes	—		
`ingested_at`	TEXT	yes	`datetime('now')`		

Unique: `(dataset_id, payload)` by index `idx_records_unique`. Index: `idx_records_dataset (dataset_id)`.

4. Model registry · `models.db` · 2 tables, 151 rows

AI models available to the data center and their versions.

`models` · 72 rows

Column	Type	Not null	Default	Key	References
`id`	INTEGER	no	—	primary	
`name`	TEXT	yes	—		
`provider`	TEXT	yes	—		
`task`	TEXT	yes	—		
`description`	TEXT	no	—		
`created_at`	TEXT	yes	`datetime('now')`		

Unique: `(name)`.

`model_versions` · 79 rows

Column	Type	Not null	Default	Key	References
`id`	INTEGER	no	—	primary	
`model_id`	INTEGER	yes	—		`models.id` on delete cascade
`version`	TEXT	yes	—		
`dimensions`	INTEGER	no	—		
`is_active`	INTEGER	yes	`0`		
`created_at`	TEXT	yes	`datetime('now')`		

Unique: `(model_id, version)`.

5. Embeddings · `embeddings.db` · 1 table, 79 rows

Vector index used for semantic search over documents.

`embeddings` · 79 rows

Column	Type	Not null	Default	Key	References
`id`	INTEGER	no	—	primary	
`document_id`	INTEGER	yes	—		
`model`	TEXT	yes	—		
`dimensions`	INTEGER	yes	—		
`vector`	TEXT	yes	—		
`created_at`	TEXT	yes	`datetime('now')`		

Unique: `(document_id, model)`. Index: `idx_embeddings_doc (document_id)`.

6. Jobs · `jobs.db` · 2 tables, 2 rows

Queue of processing work: ingest, embed, export.

`jobs` · 1 row

Column	Type	Not null	Default	Key	References
`id`	INTEGER	no	—	primary	
`kind`	TEXT	yes	—		
`payload`	TEXT	yes	`'{}'`		
`status`	TEXT	yes	`'pending'`		
`priority`	INTEGER	yes	`100`		
`submitted_at`	TEXT	yes	`datetime('now')`		

Index: `idx_jobs_status (status, priority)`.

`job_runs` · 1 row

Column	Type	Not null	Default	Key	References
`id`	INTEGER	no	—	primary	
`job_id`	INTEGER	yes	—		`jobs.id` on delete cascade
`started_at`	TEXT	yes	`datetime('now')`		
`finished_at`	TEXT	no	—		
`outcome`	TEXT	no	—		
`detail`	TEXT	no	—		

7. Audit · `audit.db` · 1 table, 466 rows

Append-only log of everything that happens in the data center.

`events` · 466 rows

Column	Type	Not null	Default	Key	References
`id`	INTEGER	no	—	primary	
`occurred_at`	TEXT	yes	`datetime('now')`		
`database`	TEXT	yes	—		
`action`	TEXT	yes	—		
`actor`	TEXT	yes	`'system'`		
`detail`	TEXT	no	—		

Index: `idx_events_db (database, occurred_at)`.

3. Datasets

The `datasets` database groups structured records into named datasets. Each record is stored as one JSON object, and each dataset keeps a schema naming its fields and their types. There are twelve datasets holding 462 records, with 94 fields between them: 54 string, 21 number, 12 integer, 6 date and 1 boolean. Every record carries every field its schema lists, though some leave a text field blank, as marked.

Dataset	Records	Fields
`business_plans`	37	6
`employment_fees`	15	8
`government_fees`	22	8
`gpu_inventory`	40	9
`incidents`	25	9
`inference_usage_daily`	90	9

Dataset	Records	Fields
`model_benchmarks`	60	9
`power_and_cooling_daily`	60	7
`service_fees`	28	7
`support_tickets`	40	4
`training_runs`	30	10
`vendor_contracts`	15	8

`business_plans` · 37 records, 6 fields

Costed business plans. Each line connects to a row in government_fees, service_fees or employment_fees through fee_code; `datacenter.plans` resolves and totals them.

`fee_code` string
`line` string
`months` integer
`notes` string (blank in 21)
`plan` string
`quantity` number

`employment_fees` · 15 records, 8 fields

Illustrative cost of employing each role: base pay, the employer tax percentage applied to it, monthly benefits and any one-off recruitment fee. Figures are invented for this data set and are not payroll or tax advice.

`base_salary_usd` number
`benefits_usd_month` number
`employer_tax_pct` number
`employment_type` string
`fee_code` string
`notes` string (blank in 9)
`recruitment_fee_usd` number
`role` string

`government_fees` · 22 records, 8 fields

Illustrative statutory and regulatory fees for operating the data centre. Jurisdictions and amounts are invented for this data set and are not legal or tax advice.

`amount_usd` number	`fee_code` string	`notes` string
`authority` string	`jurisdiction` string	`period` string
`category` string	`name` string	

`gpu_inventory` · 40 records, 9 fields

Every accelerator the data center owns, one row per card. Installed cards are keyed by host, rack and PCIe slot; cards that are spare, out for repair or retired are tracked against the STORE-A shelving, where slot is the shelf bin. last_ecc_errors is the corrected-ECC count from the most recent health sweep, and a card is pulled for RMA once it climbs past 30.

`asset_tag` string	`model` string	`slot` integer
`host` string	`purchase_date` date	`status` string
`last_ecc_errors` integer	`rack` string	`vram_gb` integer

`incidents` · 25 records, 9 fields

Every incident the operations team opened against the site, one row each, in the order they were raised. Incident ids restart at 001 each calendar year. Severity runs sev1 (tenant-visible outage) through sev4 (cosmetic or reporting only), minutes_to_resolve is exactly the gap between opened_at and resolved_at, and client_impact marks the incidents a tenant could notice without being told.

`category` string	`minutes_to_resolve` integer	`root_cause` string
`client_impact` boolean	`opened_at` string	`severity` string
`incident_id` string	`resolved_at` string	`summary` string

`inference_usage_daily` · 90 records, 9 fields

Billed inference for June 2026, one row per day per tenant across thirty consecutive days. Each tenant is served by the model named in its contract: Halvorsen Reasoning Lab on claude-sonnet-5 at 0.000006 USD per token, Tidemark Institute for Machine Learning on meta-llama/Llama-3.3-70B-Instruct at 0.0000009 USD per token, and Marigold Annotation Works on Qwen/Qwen2.5-VL-72B-Instruct at 0.0000012 USD per token. cost_usd is (input_tokens + output_tokens) x that rate, rounded to cents, so the month totals reconcile against the invoices. p95_latency_ms is the 95th percentile end-to-end request latency for the day, which tracks the average completion length and the decode rate of the serving tier; the self-hosted 70B is the slowest of the three. error_rate is the share of requests that returned a server error or timed out. Marigold's weekend volume falls away because its annotation shifts are weekdays only.

`client` string	`error_rate` number	`output_tokens` integer
`cost_usd` number	`input_tokens` integer	`p95_latency_ms` number
`date` date	`model_name` string	`requests` integer

`model_benchmarks` · 60 records, 9 fields

Bench-queue results for the models the data center serves itself, four rows per model and hardware pairing: latency_p50 and latency_p95 in milliseconds, tokens_per_second, and accuracy as a ratio. Generative rows (chat, code, vision) time a single 256-token completion behind a 1,024-token prompt, and their tokens_per_second is the aggregate decode rate, equal to batch_size x 256 x 1000 / latency_p50. Scoring rows (embedding, rerank, moderation) time one full batch, and their tokens_per_second is batch_size x the per-item token count named in the note, x 1000 / latency_p50. The hardware string gives the card and how many of them the model was sharded across. Accuracy always comes from an in-house evaluation set, named in the note, never from a published leaderboard.

- `batch_size` integer
- `hardware` string
- `measured_on` date
- `metric` string
- `model_name` string
- `notes` string
- `task` string
- `unit` string
- `value` number

`power_and_cooling_daily` · 60 records, 7 fields

Daily power and thermal readings for rack R01, the two-node H100 rack, for sixty consecutive days from 2026-06-01. avg_kw and peak_kw come from the rack PDU, inlet and outlet temperatures from the door sensors, and pue is the facility power usage effectiveness attributed to the rack for that day. Weekends show the lighter load of batch-only scheduling.

- `avg_kw` number
- `date` date
- `inlet_temp_c` number
- `outlet_temp_c` number
- `peak_kw` number
- `pue` number
- `rack` string

`service_fees` · 28 records, 7 fields

The rate card the business charges clients for AI data centre services. Referenced by business_plans through fee_code.

- `category` string
- `fee_code` string
- `minimum_units` integer
- `notes` string
- `service` string
- `unit` string
- `unit_price_usd` number

`support_tickets` · 40 records, 4 fields

Managed-support tickets and the hours logged against them.

- `category` string
- `hours` number
- `status` string
- `ticket` string

`training_runs` · 30 records, 10 fields

Fine-tuning and training jobs scheduled on the GPU fleet, one row per run. gpu_hours is the number of cards multiplied by wall-clock hours between started_at and finished_at, and cost_usd charges those hours at the flat internal rate of 2.40 USD per GPU-hour. For failed and cancelled runs, final_loss is the last value logged before the run stopped.

- `cost_usd` number
- `dataset_name` string
- `final_loss` number
- `finished_at` string
- `gpu_hours` number
- `gpus` integer
- `model_name` string
- `run_id` string
- `started_at` string
- `status` string

`vendor_contracts` · 15 records, 8 fields

Supplier agreements behind the site, one row per contract, priced per month in US dollars. Vendors are the supplier organizations in organizations.json. end_date is the current end of the agreement, and renewal_notice_days is how far ahead notice must be served. Status is read against 2026-09-20: active, pending-renewal once inside the notice window, expired once the end date has passed, renewed where a later row on this sheet supersedes the agreement, and terminated where it was ended rather than allowed to run on.

`end_date` date
`monthly_cost_usd` number
`owner_email` string
`renewal_notice_days` integer
`service` string
`start_date` date
`status` string
`vendor` string

4. Tools

Two tools operate the data center: the command line, which can read and write, and the web portal, which only reads. Both work on the same storage root.

The command line

The command line is one program, run from the repository root as `python -m datacenter <command>`. Installing the package with `pip install -e .` also gives an `sddu13` command that behaves the same way. It has 18 commands, listed below with the help text and arguments the parser itself declares. Every command also takes `-h` for its own help.

Options before the command

Option	What it does
`--version`	Show program's version number and exit.
`--root ROOT`	Where the databases live (default: ./storage).

The `SDDU13_ROOT` environment variable, when set, replaces `./storage` as the default root.

Commands

Command	What it does	Arguments and options
`init`	Create the seven databases.	—
`status`	Health and size of every database.	`--json` Machine-readable output.
`seed`	Load (or refill) the data center from its data files.	`--data DATA` Folder of JSON data files (default: datacenter/data).
`validate-data`	Check the data files without loading them.	`--data DATA` Default `datacenter/data`.
`search`	Semantic search over stored documents.	`query` Required. `--limit LIMIT` A whole number, at least 1. Default 5.

`add-document`	Store and index a document.	`title` Required.
		`--body BODY`
		`--file FILE` Read the body from this file instead.
		`--source SOURCE`
		`--tag TAG` Repeatable.

Command	What it does	Arguments and options

`add-contact`	Store a contact.	`name` Required.
		`--email EMAIL`
		`--role ROLE`
		`--phone PHONE`
		`--organization ORGANIZATION`

`contacts`	List stored contacts.	—
`documents`	List stored documents.	—
`datasets`	List stored datasets.	—
`fees`	Show the government, service and employment fee tables.	`--table TABLE` Just one of them: `employment`, `government` or `service`.
`plans`	List business plans with their totals.	—
`portal`	Serve the data center as a read-only web portal.	`--host HOST` Default 127.0.0.1. `--port PORT` Default 8013. A whole number, at least 1. `--allow-remote` Permit a non-local --host; the portal has no authentication.
`plan`	Cost one business plan line by line.	`name` Required. `--json` Machine-readable output.
`jobs`	Show the queue, or run the next job.	`--run` Run the next pending job.
`verify`	Integrity-check every database.	—
`backup`	Back up all seven databases.	`destination` Optional. Default `backups`.
`events`	Show the audit trail.	`--limit LIMIT` A whole number, at least 1. Default `20`.

The portal

`python -m datacenter portal` serves the data center as web pages on `http://127.0.0.1:8013`. It answers GET requests only and has no write path: changes go through the command line or the Python API, where the audit trail records them. It has no authentication, so it binds `127.0.0.1` and refuses a non-local `--host` unless `--allow-remote` is also passed. It serves eight fixed routes, all of them in its menu, and three more that take a name or id after the slash:

Path	Menu	Handler	What it shows
`/`	Overview	`view_overview`	One card per database with its total rows, then each database's state, size and rows per table.
`/search`	Search	`view_search`	A search box. With `?q=` it runs semantic search and lists the hits with score, title and source.
`/documents`	Documents	`view_documents`	Every stored document with its external id, source and tags.
`/contacts`	Contacts	`view_contacts`	Every contact with role, organization, email and phone.
`/datasets`	Datasets	`view_datasets`	Every dataset with its description and record count.
`/fees`	Fees	`view_fees`	The government, service and employment fee tables.
`/plans`	Plans	`view_plans`	Each business plan with its months, total, monthly average and subtotal per fee table.

Path	Menu	Handler	What it shows
`/events`	Audit	`view_events`	The most recent audit events, newest first.
`/documents/<id>`	—	`view_document`	One document: its tags, then its body with a small set of Markdown rendered after escaping. For example `/documents/1`.
`/datasets/<name>`	—	`view_dataset`	Every record of one dataset, one column per schema field. For example `/datasets/business_plans`.
`/plans/<name>`	—	`view_plan`	One plan costed line by line, with subtotals per fee table, the total and the monthly average. For example `/plans/gpu-cluster-launch`.

5. Search

Search runs on a built-in embedder, `local-hashing-v1` in `datacenter/vectors.py`, so it needs no model service and nothing installed. When a document is stored, its title and body are case-folded, NFKC-normalised and split into Unicode word tokens. Any character that is not a letter or digit separates tokens, and 121 stopwords such as `on` are dropped, so `on-call` is indexed as `call`. Each remaining token is hashed with BLAKE2b into one of 9,223,372,036,854,775,807 ($2^{63} - 1$) buckets, its count is damped to 1 + log(count), and the result is scaled to unit length. The embedding is a sparse, hashed bag of words: a map from bucket to weight that holds only the buckets the text uses, so the huge bucket space costs nothing and two different words share a bucket with a probability near 2^{-63}. Word order is not kept. The `embeddings` database holds 79 vectors, one for each of the 79 stored documents.

A query is embedded the same way, and then each of its terms is weighted by inverse document frequency in BM25's form, log(1 + (N − df + 0.5) / (df + 0.5)), where N is the number of indexed documents and df the number that contain the term: a word few documents use counts for more than one most of them use. Only the query is reweighted; stored vectors stay as they were indexed, so adding a document never means re-embedding the others. The score is the cosine similarity, which for unit vectors is the sum of products over the buckets the query and the document share. A document sharing no bucket with the query scores exactly zero and is never returned. There is no minimum score by default. The `search` command shows the top five unless given `--limit`. There is no stemming and no notion of synonyms: only the exact word form matches, so “paid” does not find “pay”. Scores rank the results of one query; they are not comparable across queries, since a long document matching a term always scores below a short one matching the same term.

Worked example

The query `network segmentation for clusters` keeps three terms (`for` is a stopword); their weights over the 79 indexed documents and the top results are below. On its own, `network` ranks “Network hardening checklist” first and “Network segmentation for GPU clusters” third. The rarer terms move it to the top.

Term	Documents with it	IDF weight
`network`	11	1.940
`segmentation`	1	3.977
`clusters`	1	3.977

#	Document	Score
1	Network segmentation for GPU clusters	0.1580
2	Network hardening checklist	0.1065
3	Training, certification and development	0.0459